Spectral Clustering and Image Segmentation

Jiwei Peng

Supervisor: Dr. Nicholas Kevlahan

July 2025

Department of Mathematics and Statistics
McMaster University

Abstract

Clustering is the act of grouping data of similar characteristics, structures, or behaviors. Any finite dataset can be transformed into an undirected, weighted simple graph by representing each data point as a vertex and assigning a weighted edge between every pair of vertices to reflect the degree of similarity between them. By analyzing the matrix representation of this similarity graph through the lenses of spectral graph theory and numerical linear algebra, one can determine an optimal partitioning of the graph that maximizes the similarity within subgraphs while ensuring the dissimilarity between them. This technique is known as spectral clustering. We provide a practical application of this method known as image segmentation, where distinct objects within an image can be detected and extracted.

1 Introduction

Clustering is one of the most studied topics in the modern world across a wide range of disciplines such as mathematics, statistics, and computer science. At its core, clustering relates to the cognitive and social instinct of human species, grouping similar objects. Early versions of clustering algorithm started to emerge in the mid-20th century of which can mostly be divided into two categories: *hierarchical clustering*, represented by single-link and complete-link, and *partitional clustering*, such as K-means [10]. Hierarchical clustering organizes data in a stepwise fashion, either by progressively merging smaller clusters or by splitting larger ones, whereas partitional clustering assigns data points to clusters simultaneously [6, 10]. For the past decades, researchers have continued to invent novel approaches while refining existing methods.

Image segmentation, in parallel, is a technique used to divide an image into desired regions based on the inherent characteristics of the image [3]. Much like clustering, it seeks to group pixels that share similar properties. One of the most impactful applications of image segmentation lies in the medical field, where it helps extract regions of interest (ROI) from medical imaging modalities to support diagnosis and treatment of diseases [3, 4].

This paper aims to provide an overview of a hierarchical clustering algorithm proposed by Shi and Malik, equipped with the normalized cut framework [13][1]. Their approach offers a mathematically intuitive method that bridges clustering to graph theory, and further to matrices via spectral graph theory. In doing so, it enables us to apply tools from numerical linear algebra to gain meaningful insight into the underlying structure of data. Beyond theoretical work, Shi and Malik demonstrated the practical application of this method in image segmentation, offering a conceptually simple yet powerful tool for computer vision tasks.

We first begin the paper with a review of some relevant background knowledge in graph theory. The paper then introduces the methodology with some intermediate level of explanations. Finally, we shift our focus to experimenting with a careful selection of grayscale images. The objective is to test the effectiveness of this method in different parameter choices and image types. In the end, we hope to make some constructive remarks on the accuracy, reliability, sensitivity, and limitations of image segmentation by spectral clustering, as well as discuss extensions and related topics.

All MATLAB code and images used in this paper can be found via GitHub .

[1] All definitions, notations, and conventions in Section 2 follow [13], unless stated otherwise.

2 Spectral Clustering Overview

2.1 *Elementary Graph Theory*

A finite **graph** $G = (V, E)$ is a non-empty set of vertices $V = \{v_1, v_2, \ldots, v_n\}$, together with a set of edges $E = \{(v_i, v_j) : 1 \leq i, j \leq n\}$ connecting certain vertices. If a graph has no edges which connect a vertex to itself (self-loop), and there are no multiple edges joining two vertices, then it is called a **simple graph**. In addition, an edge can carry a direction or weight. We say a graph is **directed** if its edges explicitly have a start point and an end point, i.e. $(v_i, v_j) \neq (v_j, v_i)$. We say a graph is **weighted** if there is an artificial value assigned to every edge in E. Throughout this paper, we will use weight(v_i, v_j) and $w_{v_i v_j}$ interchangeably to denote the weight value of an edge (v_i, v_j).

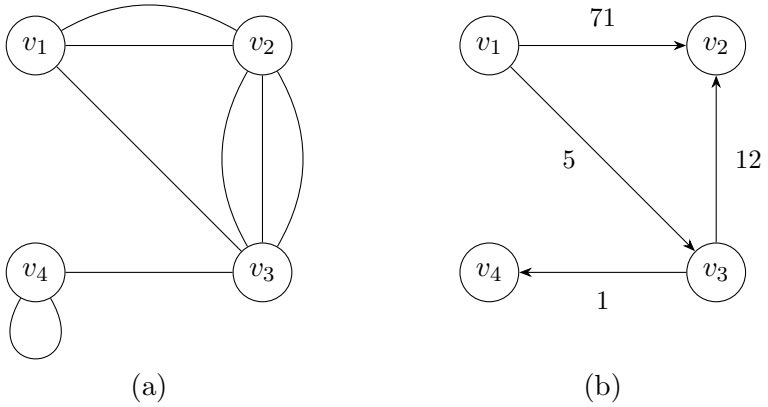


Figure 1: Comparison of graph types: (a) shows an undirected, unweighted non-simple graph, while (b) demonstrates a directed, weighted simple graph with edge weights indicated.

2.2 *Grouping Dataset as Graph Partitioning*

Consider a finite collection of data. If we represent each data point as a vertex and order them arbitrarily from 1 to n, then by forming an edge between every pair of vertices with a weight corresponding to the level of similarity between them, we can transform this dataset into an undirected, weighted simple graph $G = (V, E)$. The notion of similarity can be given arbitrarily, whether based on color, size, shape, weight, or even a combination of multiple characteristics depending on the context. For the scope of this paper, we will only consider a quantifiable and non-negative similarity, namely, weight$(i, j) \geq 0$ for all $i, j \in V$, and weight$(i, j) = 0$ if and only if i and j are not connected by an edge (i.e. they do not exhibit any similarity). The construction of similarity measure will be discussed more thoroughly in Section 3. It is also worth emphasizing that the geometric shape of such a graph is irrelevant, as the only thing we care about is the weight attribute between vertices.

More importantly, the task of dividing a dataset into two groups such that objects in the same group are very similar, while objects from different groups are not similar can be interpreted as a graph partitioning problem. We partition the vertex set of G into two disjoint sets A, B such that $A \cup B = V$ and $A \cap B = \emptyset$. We define the **cut**

value of this partition to be:

$$cut(A, B) = \sum_{i \in A, j \in B} w_{ij}. \tag{1}$$

Visually, this represents adding up the weight of every edge we have to "disconnect" in order to partition V, as illustrated in Figure 2.

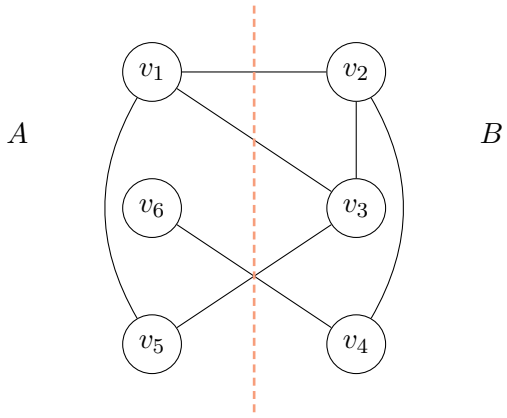


Figure 2: Visualization of a graph partitioned by a **cut**. Edge weights are hidden.

Minimizing (1) prioritizes reducing the total edge weights between A and B, which most effectively restrains the similarity between the two groups. However, this approach only completes half of the mission without recognizing the similarity within a group. For instance, one may feel tempted to assign only a single data point to A, resulting in a very small **cut** value because the fewest number of edges are "crossed". But isolating individual data points provides no meaningful partitioning in the context of clustering. To address this, we introduce a more balanced partitioning criterion: the **normalized cut**, defined as

$$Ncut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(B, A)}{assoc(B, V)} \tag{2}$$

where $assoc(A, V) = \sum_{i \in A, j \in V} w_{ij}$ is the total weight of edges connecting from vertices in A to all vertices in the graph, and $assoc(B, V)$ is similarly defined. Instead of looking at the exact value of a **cut**, the **normalized cut** considers the sum of two fractions with each not exceeding 1. Minimizing (2) not only discourages isolating small or insignificant group of vertices, but also ensures the optimal partition which accounts for both inter-group dissimilarity and intra-group similarity.

2.3 *Finding Optimal Partition*

This section is dedicated to achieving the optimal partitioning systematically through numerical linear algebra. Assume a finite graph $G = (V, E)$ such that $|V| = n$, and A, B partition V. Let x be a vector of length n consisting of only $\{-1, 1\}$. $x_i = 1$ if vertex i is in A, and -1 otherwise. x is an indicator vector that tells us how to partition a graph. Let $\boldsymbol{d}(i) = \sum_{j \in V} w_{ij}$ be the total weight of edges connecting i to all other vertices, it is usually referred to as the **degree** of vertex i. Utilizing these notations, we can rewrite (2) as:

$$Ncut(A,B) = \frac{cut(A,B)}{assoc(A,V)} + \frac{cut(B,A)}{assoc(B,V)}$$
$$= \frac{\sum_{(x_i>0,x_j<0)} -w_{ij}x_ix_j}{\sum_{x_i>0} \boldsymbol{d}_i} + \frac{\sum_{(x_i<0,x_j>0)} -w_{ij}x_ix_j}{\sum_{x_i<0} \boldsymbol{d}_i}. \tag{3}$$

Let $\mathbf{D}$ be an $n \times n$ diagonal matrix where $\boldsymbol{d}$ is on its diagonal, $\mathbf{W}$ be an $n \times n$ symmetric matrix with $\mathbf{W}_{ij} = \mathbf{W}_{ji} = w_{ij}$ and diagonal of all zeros. Moreover, let

$$k = \frac{\sum_{x_i>0} \boldsymbol{d}_i}{\sum_i \boldsymbol{d}_i}$$

and let $\vec{1} = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$ be a $n \times 1$ vector of all ones. One should recognize that

$$\sum_{x_i>0} \boldsymbol{d}_i = k \sum_i \boldsymbol{d}_i = k\vec{1}^T\mathbf{D}\vec{1} \quad \text{and} \quad \sum_{x_i<0} \boldsymbol{d}_i = (1-k) \sum_i \boldsymbol{d}_i = (1-k)\vec{1}^T\mathbf{D}\vec{1}$$

implying that

$$Ncut(A,B) = \frac{\sum_{(x_i>0,x_j<0)} -w_{ij}x_ix_j}{k\vec{1}^T\mathbf{D}\vec{1}} + \frac{\sum_{(x_i<0,x_j>0)} -w_{ij}x_ix_j}{(1-k)\vec{1}^T\mathbf{D}\vec{1}}. \tag{4}$$

Furthermore, we can rewrite $4 \times Ncut(A,B)$ as:

$$4Ncut(A,B) = \frac{(\vec{1}+x)^T(\mathbf{D}-\mathbf{W})(\vec{1}+x)}{k\vec{1}^T\mathbf{D}\vec{1}} + \frac{(\vec{1}-x)^T(\mathbf{D}-\mathbf{W})(\vec{1}-x)}{(1-k)\vec{1}^T\mathbf{D}\vec{1}} \tag{5}$$
$$= \frac{[(\vec{1}+x)-b(\vec{1}-x)]^T(\mathbf{D}-\mathbf{W})[(\vec{1}+x)-b(\vec{1}-x)]}{b\vec{1}^T\mathbf{D}\vec{1}} \tag{6}$$

where

$$b = \frac{k}{1-k} = \frac{\sum_{x_i>0} \boldsymbol{d}_i}{\sum_{x_i<0} \boldsymbol{d}_i}, \text{ suppose that } \sum_{x_i<0} \boldsymbol{d}_i \neq 0.$$

The step from (4) to (5) can be checked, however, the transformation from (5) to (6) is non-trivial, and we refer the reader to [13] for a more detailed breakdown of the mathematical manipulation.

Letting $y = (\vec{1}+x)-b(\vec{1}-x)$, we can see that $y(i) = \begin{cases} 2 & \text{if} \quad x_i > 0 \iff i \in A \\ -2b & \text{if} \quad x_i < 0 \iff i \in B \end{cases}$. y serves the exact same purpose as x by using $\{2, -2b\}$ to tell us how to partition this graph. Since $b = \frac{\sum_{x_i>0} \boldsymbol{d}_i}{\sum_{x_i<0} \boldsymbol{d}_i}$, the following relation is true:

$$y^T\mathbf{D}y = 4\sum_{x_i>0} \boldsymbol{d}_i + 4b^2 \sum_{x_i<0} \boldsymbol{d}_i = 4b \sum_{x_i<0} \boldsymbol{d}_i + 4b^2 \sum_{x_i<0} \boldsymbol{d}_i$$
$$= 4b(\sum_{x_i<0} \boldsymbol{d}_i + b\sum_{x_i<0} \boldsymbol{d}_i) = 4b(\sum_{x_i<0} \boldsymbol{d}_i + \sum_{x_i>0} \boldsymbol{d}_i)$$
$$= 4b\sum_i \boldsymbol{d}_i = 4b\vec{1}^T\mathbf{D}\vec{1}.$$

Substituting the above relation back to (6), we have:

$$4Ncut(A,B) = 4 \cdot \frac{y^T(\mathbf{D}-\mathbf{W})y}{y^T\mathbf{D}y}$$
$$\iff Ncut(A,B) = \frac{y^T(\mathbf{D}-\mathbf{W})y}{y^T\mathbf{D}y} \tag{7}$$

which takes the structure of a generalized Rayleigh quotient. We further clarify an unobvious property of y:

$$y^T\mathbf{D}\vec{1} = 2\sum_{x_i>0} \boldsymbol{d}_i - 2b\sum_{x_i<0} \boldsymbol{d}_i = 2\sum_{x_i>0} \boldsymbol{d}_i - 2\sum_{x_i>0} \boldsymbol{d}_i = 0. \tag{8}$$

If y is no longer restricted to contain only two values, but relaxed to be any arbitrary vector in $\mathbb{R}^n$ with the sole constraint (8), then (7) can be minimized by choosing y to be the eigenvector corresponding to the smallest eigenvalue of the following generalized eigenvalue problem:

$$(\mathbf{D}-\mathbf{W})y = \lambda\mathbf{D}y \tag{9}$$

which can be further rewritten as a standard eigenvalue problem as:

$$\mathbf{D}^{-\frac{1}{2}}(\mathbf{D}-\mathbf{W})\mathbf{D}^{-\frac{1}{2}}z = \lambda z \tag{10}$$

by letting

$$z = \mathbf{D}^{\frac{1}{2}}y \iff y = \mathbf{D}^{-\frac{1}{2}}z. \tag{11}$$

The two equations (9) and (10) share the same set of non-negative eigenvalues, given by the positive semi-definiteness of the Laplacian matrix $\mathbf{D}-\mathbf{W}$ and $\mathbf{D}^{-\frac{1}{2}}(\mathbf{D}-\mathbf{W})\mathbf{D}^{-\frac{1}{2}}$, but they have different eigenvectors. However, we can always use the relation between y and z as in (11) to transform the eigenvectors back and forth between those two eigensystems.

One can check that $z_0 = \mathbf{D}^{\frac{1}{2}}\vec{1}$ is an eigenvector of (10) with eigenvalue $\lambda_0 = 0$, which implies that $y_0 = \vec{1}$ minimizes our target equation (9). However, such y_0 directly translates to trivial partitioning $V = V \cup \emptyset$, and it violates constraint (8). Thus, we proceed to the next minimizer vector y_1 which corresponds to the second smallest eigenvalue λ_1. Putting everything together, our objective is to find the second smallest eigenvector y_1 of the generalized eigenvalue problem (9), which is the unique non-trivial solution of the optimal partitioning problem solved by the **normalized cut**[2].

3 Image Segmentation

3.1 *Theoretical Background*

A straightforward and useful implementation of spectral clustering is image segmentation. The simplest case to work with is a grayscale image. In any computer application, a grayscale image is processed as a single matrix with its entries reflecting the brightness level of each pixel from 0 (white) to 255 (black). Suppose a grayscale

[2]The proof of the statement presented in the last two paragraphs of 2.3 is not provided by [13], and is rather complicated. Our report aims to fill this gap; see 4 for the proof.

image has size m by n, then we can repeat the process discussed in 2.2 by representing every pixel as a vertex and assigning a weighed edge between each pair of pixels to indicate their similarity, and therefore obtaining a weighted simple graph. The pairwise similarity in this context relates to the possibility of two pixels belonging to the same object/segment and is determined by their brightness level and spatial location.

The construction of an appropriate similarity graph is among the most imperative parts of spectral clustering. There are several popular ways to do so. *The ϵ-neighborhood graph* and *(mutual) k neighbor graphs* are local discrete methods that work with only one parameter, while treating nearby vertices with equal importance [14]. They are primarily applicable when we want to cluster a dataset solely according to its spatial distribution. In contrast, *the fully connected graph* recognizes global similarity, and an example is the Gaussian similarity function which provides a continuously varying similarity between data points as their distance changes.

In our more complicated case of image segmentation, we will adopt an extension of the Gaussian similarity function as follows:

$$w_{ij} = \begin{cases} e^{\frac{-\|F(i)-F(j)\|_2^2}{\sigma_I^2}} \cdot e^{\frac{-\|X(i)-X(j)\|_2^2}{\sigma_X^2}} & \text{if} \quad \|X(i)-X(j)\|_2 < r \\ 0 & \text{otherwise} \end{cases} \tag{12}$$

where the brightness level (F) and spatial location (X) are inputs, and σ_I, σ_X, r are positive real value parameters [13]. This formulation reflects the intuition that the weights depend on both the distance between pixels and their difference in brightness level. If two pixels are of small spatial proximity and have similar brightness values, then we admit a higher likelihood of them belonging to the same segment, and the opposite holds true as well.

The parameters involved in (12) also have profound implications. r can be interpreted as the distance threshold - once two pixels exceed a certain distance, we no longer consider the possibility of them belonging to the same object. It "cuts off" the infinite span of (12) in $\mathbb{R}^3$. $\sigma = \{\sigma_I, \sigma_X\}$ controls the shape of (12). Smaller magnitude of σ creates a steep decent after the peak, giving rise to a stricter condition for two pixels to be similar, whereas larger magnitude of σ has a contrary effect.

Unfortunately, no systematic study has been conducted to provide guidance on finding optimal parameter choices. Perturbations in parameters can drastically alter the outcome of segmentation, and so the partition may not be stable [14]. Our experiments suggest that parameter selection is heavily empirical, and the best result is usually achieved through trials and errors, as we will show later.

3.2 *Experiments*

Since similarity is symmetric and our similarity function (12) introduces the cutoff value r that prevents distant pixels from being similar, we are offered the opportunity to construct the adjacency matrix $\mathbf{W}$ in a computationally efficient manner. In particular, we only need to consider a local square neighborhood of side length $2r$ around each pixel. This square fully covers the pixel's l_2 neighborhood of radius r. By scanning only the right-hand portion of this neighborhood and updating two positions of $\mathbf{W}$ simultaneously, we can generate the matrix using slightly fewer than $O(mn(2r+1)(r+1))$ operations, much less than that of the direct method which requires $O(m^2n^2)$ operations.

Le *et al.* proposed a parameter set

$$\{r = 11, \sigma_X = 8, \sigma_I = 17.85\}$$

that performs well across different image types with resolutions below 200×300 pixels [11]. This paper will rely extensively on this set of parameters in the subsequent analysis.

The first example we present is a grayscale image of a baby from Shi [12], consisting of 104×106 pixels. This image contains a dominant object with clear boundary and a high contrast against the background, making it inherently conductive to segmentation. Therefore, it is relatively robust to parameter perturbations.

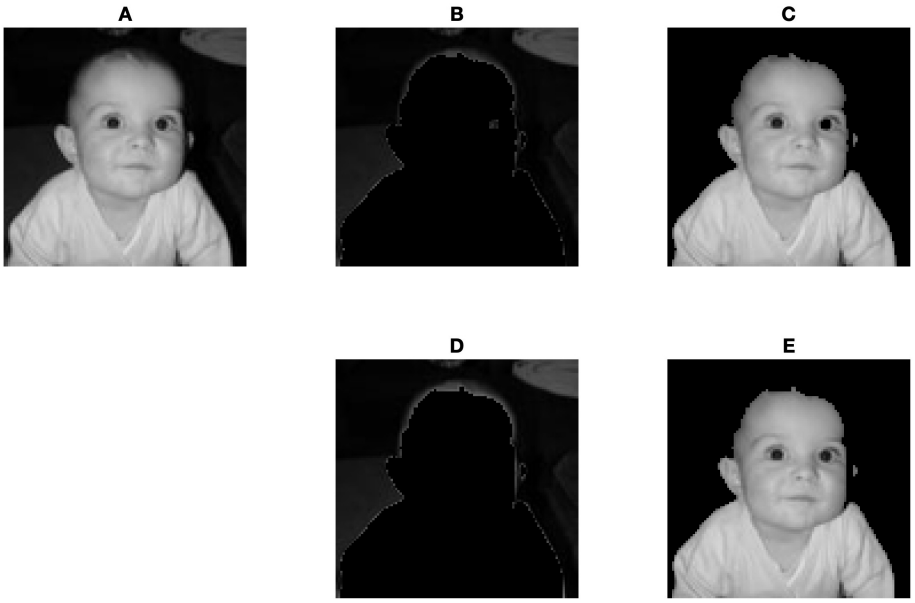


Figure 3: Result of segmentation of the original image **A**: **B** and **D** contain all vertices whose corresponding entry in y_1 is greater than zero; **C** and **E** contain all vertices whose corresponding entry in y_1 is smaller than or equal to zero. Parameter setting for the two segments in the **top**: $r = 11, \sigma_X = 8, \sigma_I = 17.85$. Parameter setting for the two segments in the **bottom**: $r = 7, \sigma_X = 5, \sigma_I = 25$.

We remind the reader from Section 2.3 that, theoretically, the partition vector y should serve the purpose of an indicator. More specifically, it should only take on two discrete values, one greater than zero and the other smaller than zero. In this way, we can use the sign of each entry to determine whether to assign a vertex to one group or the other. However, in practice, the second smallest eigenvector y_1 often consists of floating point numbers of different values. For example, we may have $y_1 = \begin{bmatrix} 0.667 & -0.775 & 0.200 & -0.043 & \cdots & -0.001 \end{bmatrix}^T$, which does not explicitly guide us partitioning the graph. The natural approach to address this issue is to use zero as the splitting point, following the sign-based rationale above. In other words, we assign a vertex to A if its corresponding entry in y_1 is greater than zero, and we assign a vertex to B if its corresponding entry in y_1 is smaller than or equal to zero. This is adopted in Figure 3. But a more thoughtful strategy is to iterate over a finite number of evenly spaced points chosen from the interval formed by the smallest and largest entries of y_1. The optimal splitting point is then selected as the

one that yields a partition minimizing the $Ncut$ value [13]. Switching to the optimal splitting point may not bring obvious improvements when parameters already provide satisfying segmentation. Thus, we consider a defective set of parameters to illustrate the effect, as in Figure 4

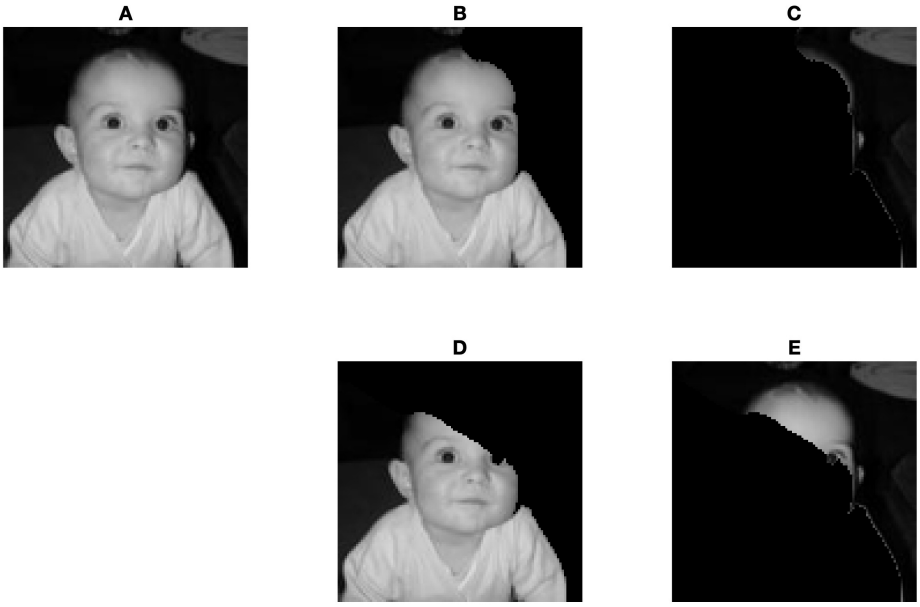


Figure 4: Comparison of segmentation of the original image **A** with two choices of splitting point: **top** two segments are attained by using the optimal splitting point $\approx -1.09 \times 10^{-3}$; **bottom** two segments are attained by using zero as the splitting point. Parameter setting: $r = 4, \sigma_X = 5, \sigma_I = 25$.

There are two major advantages of this optimal splitting point. Firstly, it provides a more accurate segmentation. Secondly, it plays a crucial role in rescuing the cohesiveness of an object within the image. As showed in Figure 4, when we shrink the value of r, and 0 as the splitting point provides a partition that cuts through the object, our optimal splitting point still succeeds.

If one is interested in understanding how perturbations in the parameters, particularly σ_I, σ_X, affect the segmentation outcome, this can be analyzed using a normalized difference. We now provide the setup. Let $\Delta\sigma = \{\Delta\sigma_I, \Delta\sigma_X\}$ be parameter perturbations, and define the perturbed set as $\sigma + \Delta\sigma = \{\sigma_I + \Delta\sigma_I, \sigma_X + \Delta\sigma_X\}$. By performing spectral clustering with both σ and $\sigma + \Delta\sigma$, we obtain two partitions respectively. We can then define the following:

$$y_{1,\sigma}(i) = \begin{cases} 1 & \text{if} \quad y_1(i) > \text{splitting point w.r.t. partition using } \sigma \\ -1 & \text{if} \quad y_1(i) \leq \text{splitting point w.r.t. partition using } \sigma \end{cases}$$

and

$$y_{1,\sigma+\Delta\sigma}(i) = \begin{cases} 1 & \text{if} \quad y_1(i) > \text{splitting point w.r.t. partition using } \sigma + \Delta\sigma \\ -1 & \text{if} \quad y_1(i) \leq \text{splitting point w.r.t. partition using } \sigma + \Delta\sigma \end{cases}$$

whether the splitting point is set to zero or chosen optimally.

This technique is known as **thresholding**. We define the difference vector to be $v_d = y_{1,\sigma+\Delta\sigma} - y_{1,\sigma}$ and we can normalize it to be a number between 0 and 1.

Mathematically, we have

$$\text{norm}(v_d) = \frac{\|\frac{1}{2}v_d\|^2}{mn}, \quad 0 \leq \text{norm}(v_d) \leq 1.$$

Here, $\text{norm}(v_d) = 0$ indicates that every vertex has been assigned to the same region in both partitions, while $\text{norm}(v_d) = 1$ corresponds to a complete disagreement. However, one should be careful that $\text{norm}(v_d) = 1$ still implies two identical partitions, with the extracted regions swapping with each other. Therefore, we consider the absolute normalized difference defined as: $|\text{norm}(v_d)| = \min\{\text{norm}(v_d), 1-\text{norm}(v_d)\}$. The two partitions presented in Figure 3 have a normalized difference: $\text{norm}(v_d) \approx 0.0104$, providing minimal visual difference.

The second image we examine is a grayscale flower of 135×240 pixels [9]. Although the main object of the image is surrounded by some noise in the background, it shares structural similarities with the baby image: central dominant object, distinct boundaries, and weaker but still noticeable contrast. Consequently, the image shows robustness to parameter perturbations as well.

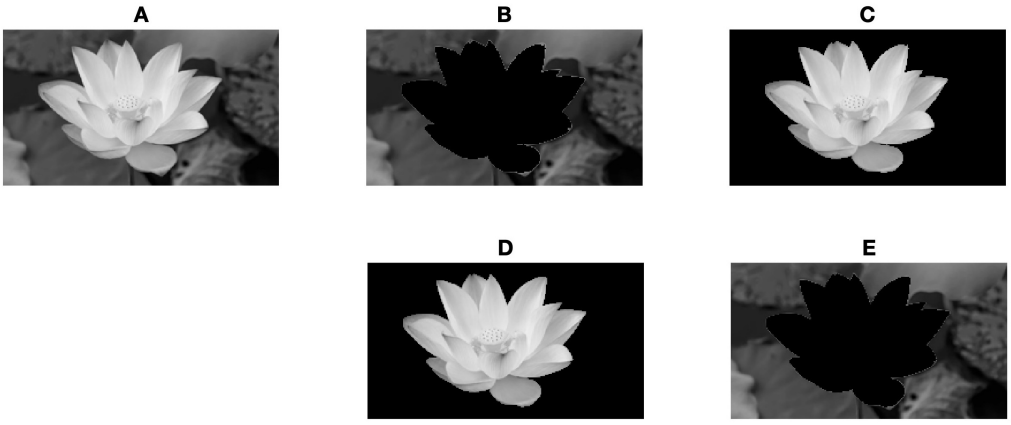


Figure 5: Result of segmentation of the original image **A**: the respective optimal splitting point is used. Parameter setting for the two segments in the **top**: $r = 11, \sigma_X = 8, \sigma_I = 17.85$. Parameter setting for the two segments in the **bottom**: $r = 8, \sigma_X = 7, \sigma_I = 13$. The normalized difference between the two partitions is $\text{norm}(v_d) \approx 0.9973$, which translates to an absolute normalized difference $|\text{norm}(v_d)| \approx 0.0027$, smaller than that of Figure 3.

Lastly, we present a grayscale image of a dog of size 128×128 whose segmentation proves challenging and requires manual intervention [8]. Although it is technically difficult to exhaust all possible parameter combinations, sufficient experimentation reveals a consistent issue: at least half of the background remains attached to the object across a variety of parameter choices. To fully isolate the object, we perform a straightforward clustering algorithm as follows.

We first perform all necessary steps for the spectral segmentation of the image. Then we scatter plot the second smallest eigenvector y_1 by placing the position of each entry on the x-axis and its exact value on the y-axis. MATLAB automatically normalizes y_1 during computational steps to prevent numerical instability. Followed by that, we can iterate over a finite number of horizontal bands with equal width to capture the cluster moly likely represents the object. This is evaluated on the basis of both comparative and absolute advantage in density. Our target band should be

denser than neighboring bands while at the same time belonging to one of the densest clusters globally. Finally, we choose either the upper or lower bound of the target band as the second splitting point and extract the object accordingly. The exercise of the proposed clustering method depends on visual information gained from the scatter plot. In our case, we iterate evenly spaced bands downward from the optimal splitting point and adopt the lower bound of our target band as the second splitting point. Looking at the vertical axis labels, as well as the distribution of data points in Figure 7, we choose the band width to be 2.5×10^{-4} to capture the narrow horizontal cluster trapped between $\mathbf{entry = 0}$ and $\mathbf{entry = -5 \times 10^{-4}}$ that we speculate to be the data representation of the dog.

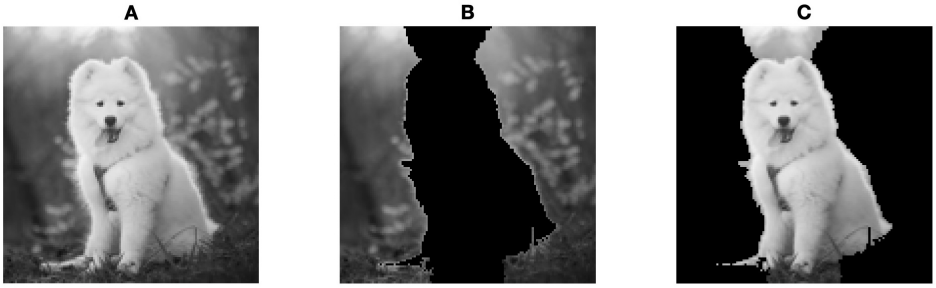


Figure 6: The original image **A** is defensive to the standard method of segmentation. We use two splitting points to attain the segments. **B** corresponds to all vertices whose entry in y_1 is larger than the optimal splitting point, or smaller than the second splitting point. **C** corresponds to all vertices whose entry in y_1 is smaller than or equal to the optimal splitting point, and larger than or equal to the second splitting point. Parameter setting: $r = 11, \sigma_X = 8, \sigma_I = 17.85$.

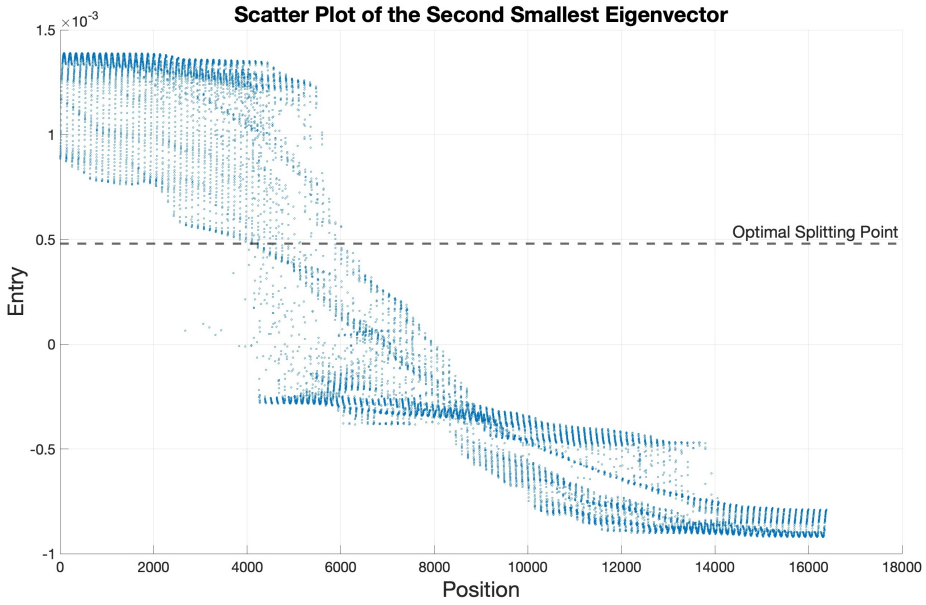


Figure 7: Scatter plot of the second smallest eigenvector y_1 of the dog image; x-axis represents the position of each entry in the vector and y-axis represents its value. We iterate downward from the optimal splitting point to achieve the second splitting point.

Unfortunately, with the techniques covered in this paper, the small piece of background attaching the top of the dog's head in **C** in Figure 6 cannot be removed perfectly unless sacrificing the completeness of the dog. This phenomenon is evident in the scatter plot of the second smallest eigenvector y_1 in Figure 7, where the data distribution is highly connected without major gaps, giving rise to a intertwine between the dog and background pixels. Thus, slightly altering the splitting point may shrink the undesired white region in the background, but it also omits pixels belonging to the dog. We present some segmentation results of different choices of splitting points in Figure 8 for comparison.

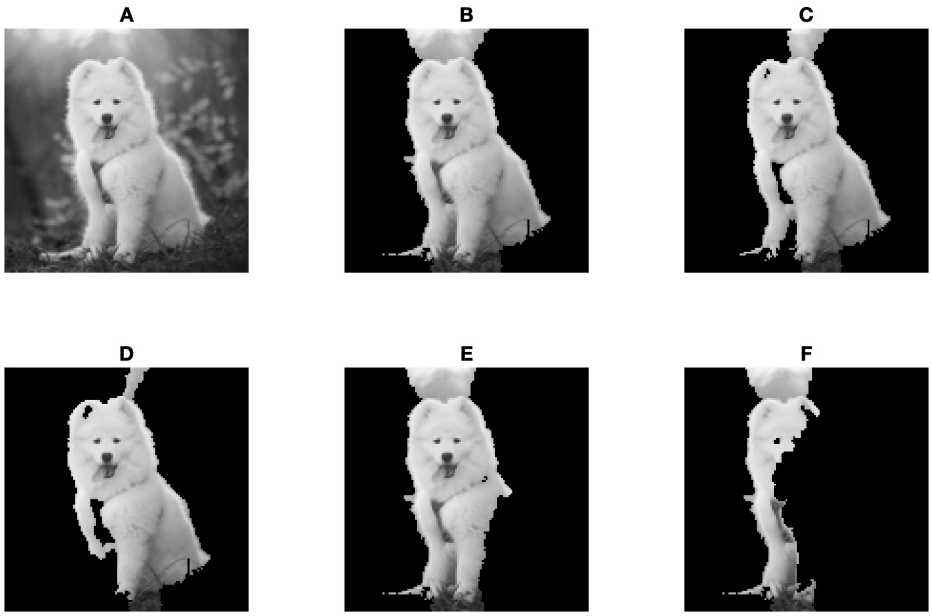


Figure 8: Result of segmentation of the original image **A**: **B** shows the segmentation from Figure 6; **C** and **D** are obtained by gradually lowering the first (optimal) splitting point while keeping the second splitting point unchanged; **E** and **F** are attained by gradually increasing the second splitting point gradually while keeping the first (optimal) splitting point unchanged.

This artifact occurs because the measure of similarity constructed in 12 only considers the brightness level and spacial proximity of pixels. Since the top white region shares similar brightness with the dog while being spatially adjacent to it. It is not surprising that our algorithm overlooks this imprecision.

4 Conclusion and Discussion

This paper explored spectral clustering with the normalized cut framework, highlighting its theoretical foundation and practical application in image segmentation. We found this approach to be remarkably accessible: users can segment objects from personal images, on their own computer using manageable MATLAB code, without relying on proprietary software developed by large companies.

However, like all existing models in the world, the use of spectral clustering in image segmentation is not without limitations. We intend to discuss a few important ones. The most significant drawback is the empirical nature of parameter selection

when constructing the similarity function (12). In most cases, the effectiveness of a combination of parameters cannot be predicted until the result of segmentation is observed. The inevitable time-consuming process of trying different parameter values may discourage people from embracing this method. Secondly, examining only the brightness level and spatial location of pixels omits crucial information that may not be reflected directly by those two factors. For example, it is unlikely for our algorithm to be equipped with the ability to detect an object that shares an identical color with its surroundings. Finally, multi-step segmentation of an image containing several objects proves technically difficult. In our framework, each of the two segments represents the region it does not possess as zero pixels. Consequently, reapplying our algorithm to each segment once more simply groups all zero pixels together, rather than isolating new objects.

A theoretical or computational way to alleviate any of the issues mentioned above could be an interesting and meaningful direction to explore for future research. Segmentation through texture is a rich and well-established area that focuses on distinguishing regions of different textures within an image [2]. For example, the texture of a piece of wood differs noticeably from that of a rock. Traditional ways of texture segmentation involve pairing a texture feature extraction technique with a proper segmentation method [2]. An integration between our method and a feasible texture segmentation algorithm may prevent grouping distinct objects with similar brightness.

Despite these drawbacks, spectral clustering remains a valuable tool when applied under suitable conditions. In summary, the class of low-resolution grayscale images that contain a single, well-defined object with high contrast against the background proves ideal. Outside of this scope, the method may struggle to produce reliable results without further modification, such as the eigenvector density clustering method we proposed at the end of Section 3.2

Beyond image segmentation, spectral clustering finds application in other domains such as documents and words co-clustering, and face clustering in videos. Instead of following the traditional trajectory of clustering documents based on specific word frequencies, or clustering words according to their tendency to occur in the same document, spectral clustering can be adapted to perform the two tasks simultaneously [7]. On the other hand, one can apply spectral clustering to identify facial images belonging to the same actor/actress in a video [1].

Appendix

Lemma 1. *The Laplacian matrix of an undirected, weighted simple graph is real symmetric and positive semi-definite.*

Proof. $\mathbf{D}$ is diagonal and $\mathbf{W}_{ij} = \mathbf{W}_{ji}$, therefore $\mathbf{L} = \mathbf{D} - \mathbf{W}$ is also symmetric. Note that

$$\begin{aligned} v^T\mathbf{L}v = v^T\mathbf{D}v - v^T\mathbf{W}v &= \sum_i \boldsymbol{d}_i v_i^2 - \sum_{i,j} w_{ij}v_iv_j \\ &= \sum_{i,j} w_{ij}v_i^2 - \sum_{i,j} w_{ij}v_iv_j \\ &= \left(\frac{1}{2}\sum_{i,j} w_{ij}v_i^2 + \frac{1}{2}\sum_{i,j} w_{ij}v_j^2\right) - \sum_{i,j} w_{ij}v_iv_j \\ &= \frac{1}{2}\sum_{i,j} w_{ij}(v_i - v_j)^2 \geq 0 \end{aligned}$$

so $\mathbf{L}$ is positive semi-definite. This proof is inspired by [14]. □

From linear algebra, we know that $\mathbf{L}$ of size $n \times n$ is real symmetric ensures all eigenvalues of $\mathbf{L}$ are real, and $\mathbf{L}$ has n orthonormal eigenvectors that form a basis of $\mathbb{R}^n$. $\mathbf{L}$ being positive semi-definite further informs us that its eigenvalues are non-negetive. We denote the set of orthonormal eigenvectors of $\mathbf{L}$ as $\{v_1, v_2, \cdots, v_n\}$ where $0 \leq \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$.

Let u be any arbitrary vector in $\mathbb{R}^n$ of the form $u = c_1v_1 + c_2v_2 + \cdots + c_nv_n$ where $c_i \in \mathbb{R}$. Since $v_i^Tv_j = 0 \;\forall\; i \neq j$ and $v_i^Tv_j = 1$ if $i = j$, the Rayleigh quotient between u and $\mathbf{L}$ can be rewritten as:

$$\frac{u^T\mathbf{L}u}{u^Tu} = \frac{\sum_{i=1}^n c_i^2\lambda_i}{\sum_{i=1}^n c_i^2} = \frac{c_1^2}{\sum_{i=1}^n c_i^2}\lambda_1 + \frac{c_2^2}{\sum_{i=1}^n c_i^2}\lambda_2 + \cdots + \frac{c_n^2}{\sum_{i=1}^n c_i^2}\lambda_n$$

which can be interpreted as a weighted average (or convex combination, more precisely) of all eigenvalues of $\mathbf{L}$. It attains its minimum value λ_1 when $c_1 \neq 0$, and $c_2 = c_3 = \cdots = c_n = 0$. If we impose an additional constraint on u to make it perpendicular to the direction of v_1, then we should expect

$$\min\frac{u^T\mathbf{L}u}{u^Tu} = \lambda_2 \text{ if } u^Tv_1 = \langle u, v_1\rangle = 0$$

by choosing $c_2 \neq 0, c_3 = c_4 = \cdots = c_n = 0$. This result is presented by [5] in Chapter 4.

The above idea naturally extends to the notion of a generalized Rayleigh quotient[3]. If we assume the degree matrix $\mathbf{D}$ to be non-singular, then

$$\begin{aligned} Ncut(A,B) = \frac{y^T(\mathbf{D}-\mathbf{W})y}{y^T\mathbf{D}y} &= \frac{y^T\mathbf{L}y}{y^T\mathbf{D}^{\frac{1}{2}}\mathbf{D}^{\frac{1}{2}}y} = \frac{y^T\mathbf{L}y}{(\mathbf{D}^{\frac{1}{2}}y)^T(\mathbf{D}^{\frac{1}{2}}y)} \\ &= \frac{(\mathbf{D}^{-\frac{1}{2}}z)^T\mathbf{L}(\mathbf{D}^{-\frac{1}{2}}z)}{z^Tz} = \frac{z^T(\mathbf{D}^{-\frac{1}{2}}\mathbf{L}\mathbf{D}^{-\frac{1}{2}})z}{z^Tz} \end{aligned} \tag{13}$$

[3]The following proof is an original work by the author.

by denoting $z = \mathbf{D}^{\frac{1}{2}} y \iff y = \mathbf{D}^{-\frac{1}{2}} z$, and $\mathbf{D}^{\frac{1}{2}}, \mathbf{D}^{-\frac{1}{2}}$ are the element-wise operations of taking the square roots/reciprocal of the square roots of the diagonal of $\mathbf{D}$. One can check that $\mathbf{D}^{-\frac{1}{2}}\mathbf{L}\mathbf{D}^{-\frac{1}{2}}$ is also positive semi-definite, and its smallest eigenvalue is 0 with corresponding eigenvector being $\mathbf{D}^{\frac{1}{2}}\vec{1}$. However, from (8), we know

$$\begin{aligned} & y^T \mathbf{D}\vec{1} = 0 \\ \implies & (\mathbf{D}^{-\frac{1}{2}} z)^T \mathbf{D}\vec{1} = z^T \mathbf{D}^{-\frac{1}{2}} \mathbf{D}\vec{1} = z^T \mathbf{D}^{\frac{1}{2}}\vec{1} = 0. \end{aligned}$$

This is exactly the perpendicular constraint we defined earlier in the proof. Accordingly, the second smallest eigenvalue of $\mathbf{D}^{-\frac{1}{2}}\mathbf{L}\mathbf{D}^{-\frac{1}{2}}$ minimizes (13), by forcing z to point in the direction of the eigenvector corresponding to the second smallest eigenvalue. Lastly, we can calculate y by $y = \mathbf{D}^{-\frac{1}{2}} z$.

References

[1] C. Chrysouli, N. Vretos, and I. Pitas, *Face clustering in videos based on spectral clustering techniques*, The first asian conference on pattern recognition, 2011, pp. 130–134.

[2] M. F. A. Fauzi and P. H. Lewis, *Automatic texture segmentation for content-based image retrieval application*, Pattern Analysis and Applications **9** (2006), no. 4, 307–323.

[3] J. Q. Gao, B. B. Wang, Z. Y. Wang, Y. F. Wang, and F. Z. Kong, *A wavelet transform-based image segmentation method*, Optik **208** (2020).

[4] Y. H. Guo and A. S. Ashour, *Neutrosophic sets in dermoscopic medical image segmentation*, Neutrosophic set in medical image analysis, 2019, pp. 229–243.

[5] R. A. Horn and C. R. Johnson, *Matrix analysis*, 2nd ed., Cambridge University Press, 2012.

[6] L. Hubert, *Approximate evaluation techniques for the single-link and complete-link hierarchical clustering procedures*, Journal of the American Statistical Association **69** (1974), no. 347, 698–704.

[7] D. Inderjit S., *Co-clustering documents and words using bipartite spectral graph partitioning*, Kdd01: Acm sigkdd international conference on knowledge discovery and data mining, 2001, pp. 269–274.

[8] Internet Image of a Dog. https://www.thepioneerwoman.com/home-lifestyle/pets/g35482456/medium-sized-dog-breeds/.

[9] Internet Image of a Lotus Flower. https://www.planetnatural.com/lotus-flower/.

[10] A. K. Jain, *Data clustering: 50 years beyond k-means*, Patterns Recognition Letters **31** (2010), no. 8, 651–666.

[11] T. V. Le, C. A. Kulikowski, and I. B. Muchni, *A graph-based approach for image segmentation*, Advances in visual computing, 2008, pp. 278–287.

[12] J. B. Shi, *Normalized cut image segmentation and clustering code*. https://www.cis.upenn.edu/~jshi/software/.

[13] J. B. Shi and J. Malik, *Normalized cuts and image segmentation*, IEEE Transactions on Pattern Analysis and Machine Intelligence **22** (2000), no. 8, 888–905.

[14] U. von Luxburg, *A tutorial on spectral clustering*, Statistics and Computing **17** (2007), no. 4, 395–416.